The Kalman Filter

Signal Extraction from Noisy Financial Data

IEOR E4576 — Data-Driven Models in Finance — Fall 2026 — Week 5

Extracted from *Machine Learning for Finance*, Stefan Jansen (Packt) — Chapter 4, pp. 100–103

Contents

1 Motivation: Noise in Financial Data **1**

2 How the Kalman Filter Works **1**
2.1 Graphical Model . . . 1
2.2 Assumptions . . . 2
2.3 Mathematical Formulation . . . 2
2.4 Two-Step Algorithm . . . 2
2.5 Advantages and Disadvantages . . . 2

3 Applying the Kalman Filter with `pykalman` **2**
3.1 Univariate Example: Smoothing a Price Series . . . 2
3.2 Tuning Q and R . . . 3
3.3 Kalman Smoother (Offline) . . . 3
3.4 Parameter Estimation via EM . . . 4

4 Summary **4**

1 Motivation: Noise in Financial Data

The concept of **noise** in data relates to the domain of signal processing, which aims to retrieve the correct information from a signal. As a signal moves through a medium, environmental interference can be added in the form of noise, making it necessary to separate the two.

The Kalman filter was introduced in 1960 and has become very popular for many applications that require processing noisy data, because it permits more accurate estimates of the underlying signal. The technique is widely used to:

- track objects in computer vision,
- support localisation and navigation of aircraft and spacecraft,
- control robotic motion based on noisy sensor data, and
- analyse time series in finance and data science.

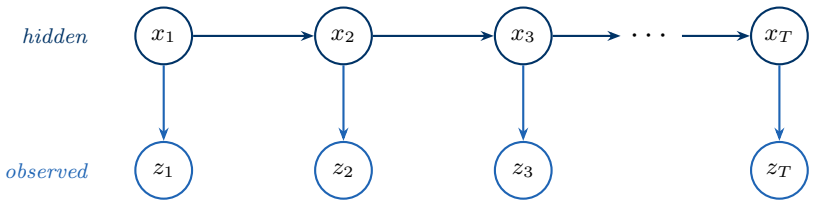
In finance, *noise* implies that raw data contains useful information — trading signals, true prices — mixed with irrelevant, extraneous variation. Separating them is non-trivial because the true signal is not directly observable.

2 How the Kalman Filter Works

The Kalman filter is a **dynamic linear model** of sequential data that *adapts to new information as it arrives*. Rather than using a fixed-size window (like a moving average) or a set of fixed weights (like an EMA), it incorporates each new observation via a probabilistic model.

2.1 Graphical Model

The filter defines a probabilistic model over **hidden states** $x_1, x_2, \ldots, x_T$ (the true unobserved price) and **observations** $z_1, z_2, \ldots, z_T$ (the noisy close prices we measure):



Each observation z_t depends only on the current hidden state x_t (downward arrows); each hidden state depends only on the previous one — the **Markov property** (horizontal arrows). The filter takes a **Bayesian approach**, propagating the posterior $p(x_t \mid z_1, \ldots, z_t)$ forward in time.

2.2 Assumptions

- The system behaves **linearly**.
- The hidden state is a **Markov chain**: x_t depends only on x_{t-1}.
- Noise is **Gaussian and uncorrelated** with constant covariance.

2.3 Mathematical Formulation

The model is defined by two core equations and an initial condition:

$$x_0 \sim \mathcal{N}(\mu_0, \Sigma_0) \qquad (\texttt{initial_state_mean}, \texttt{initial_state_covariance})$$

$$\begin{aligned} x_{t+1} &= A_t x_t + b_t + \epsilon_t^{(1)}, \qquad \epsilon_t^{(1)} \sim \mathcal{N}(0, Q) \quad \text{(state transition, } \texttt{transition_covariance}\ Q) \\ z_t &= C_t x_t + d_t + \epsilon_t^{(2)}, \qquad \epsilon_t^{(2)} \sim \mathcal{N}(0, R) \quad \text{(observation, } \texttt{observation_covariance}\ R) \end{aligned} \tag{1}$$

In the univariate random-walk model used here: $A_t = C_t = 1$, $b_t = d_t = 0$.

2.4 Two-Step Algorithm

Prediction step

Propagate the prior estimate and its uncertainty forward:

$$\hat{x}_{t|t-1} = A\hat{x}_{t-1}, \qquad P_{t|t-1} = AP_{t-1}A^\top + Q$$

Update step

Incorporate the new observation z_t via the **Kalman gain** K_t, which weights the prediction against the observation by their relative certainties:

$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + R}, \qquad \hat{x}_t = \hat{x}_{t|t-1} + K_t\big(z_t - \hat{x}_{t|t-1}\big), \qquad P_t = (1 - K_t)P_{t|t-1}$$

2.5 Advantages and Disadvantages

Advantages

Adapts to non-stationary data; processes each observation in $O(1)$; propagates uncertainty alongside the state estimate; no fixed window required.

Disadvantages

Assumes linearity and Gaussian noise — assumptions financial data often violates. Extensions (Extended KF, Unscented KF, particle filter) address these.

3 Applying the Kalman Filter with `pykalman`

The filter is particularly useful for **rolling estimates** of values or model parameters that change over time, as it adapts at every step and naturally weighs recent observations more heavily — without a fixed window length.

3.1 Univariate Example: Smoothing a Price Series

The textbook example applies the filter to the S&P 500 close price series for 2008–09 (pp. 102–103). The same pattern applies directly to a 1-minute close price series.

Model (random walk + noise):

$$x_{t+1} = x_t + w_t, \quad w_t \sim \mathcal{N}(0, Q) \qquad \text{(hidden true price)} \qquad z_t = x_t + v_t, \quad v_t \sim \mathcal{N}(0, R) \qquad \text{(observed close)}$$

Step 1 — Initialise:

```
from pykalman import KalmanFilter

kf = KalmanFilter(
    transition_matrices      = [1],    # x_{t+1} = 1 * x_t + noise
    observation_matrices     = [1],    # z_t     = 1 * x_t + noise
    initial_state_mean       = 0,
    initial_state_covariance = 1,
    observation_covariance   = 1,      # R: measurement noise
    transition_covariance    = 0.01,   # Q: process noise
)
```

Step 2 — Run the forward filter:

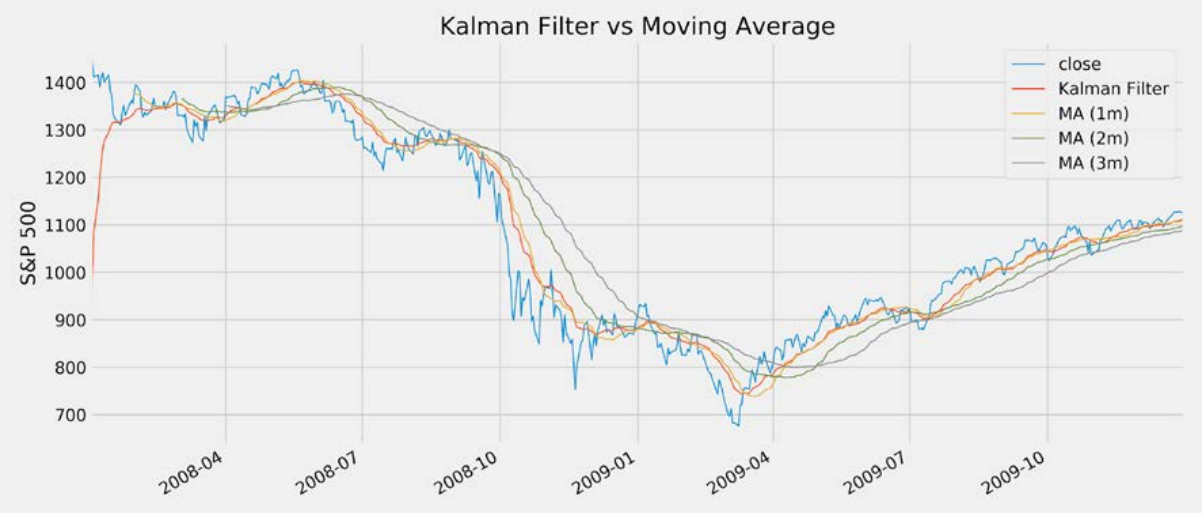
```
state_means, state_covs = kf.filter(prices)
# state_means : estimated hidden state x_t at each bar
# state_covs  : posterior variance P_t (uncertainty band)
```

Step 3 — Compare with moving averages:

```
result = prices.to_frame('close')
result['Kalman Filter'] = state_means
for months in [1, 2, 3]:
    result[f'MA ({months}m)'] = prices.rolling(window=months * 21).mean()

result.plot(title='Kalman Filter vs Moving Average', figsize=(14, 6), lw=1)
```

The Kalman filter performs similarly to a 1-month moving average but is *more sensitive to changes in the behaviour of the time series*, because its effective window adapts dynamically rather than remaining fixed.



Kalman filter vs. moving average on daily S&P 500 prices (2008–09). The Kalman filter (orange) responds more quickly to trend changes than the 2- and 3-month MAs while remaining smoother than the 1-month MA. Source: Jansen, *Machine Learning for Finance*, Fig. 4.6.

3.2 Tuning Q and R

The ratio Q/R determines the steady-state Kalman gain and controls the smoothing–responsiveness trade-off:

Parameter	Small	Large
Q (process noise)	Slow-drifting state ⇒ heavy smoothing	Fast-changing state ⇒ tracks closely
R (measurement noise)	Trust observations ⇒ tracks closely	Distrust observations ⇒ heavy smoothing

3.3 Kalman Smoother (Offline)

The forward filter is *causal*. The **smoother** adds a backward pass for better retrospective estimates (not usable in real time):

```
state_means_s, state_covs_s = kf.smooth(prices)
```

3.4 Parameter Estimation via EM

To avoid hand-tuning Q and R, `pykalman` supports fitting them from data via Expectation-Maximisation:

```
kf_em = kf.em(prices, n_iter=10)
print(kf_em.transition_covariance)   # estimated Q
print(kf_em.observation_covariance)  # estimated R
```

4 Summary

Concept	Description
Hidden state x_t	Unobserved "true" price or signal
Observation z_t	Noisy market price
Kalman gain K_t	Weight on new observation vs. prior
Forward filter	Causal; real-time applicable
Smoother	Non-causal; better retrospective estimate
EM fitting	Data-driven estimation of Q and R

The Kalman filter is more **adaptive** than a fixed moving average, and the ratio Q/R controls its smoothing–responsiveness trade-off. It propagates **uncertainty** P_t alongside the estimate, so the posterior covariance widens during volatile periods. When there is no prior knowledge of Q and R, EM can estimate them from data.